Seção 1: Identificação

Lucca Sardá Tonin Matos Matrícula: 21104811

Marianne Gaertner Matrícula: 21200747

Laboratório 2 Data: 14/10/2025

Seção 2: Descrição da Atividade 1:

Na Atividade 1 deste laboratório, foi proposto o desenvolvimento de um kernel, distinto dos kernels normalizado e gaussiano, capaz de reduzir digitalmente os traços de uma tatuagem, conforme ilustrado na Figura 1.



Figura 1: Imagem a ser tratada digitalmente.

Seção 2.1: Solução Proposta para a Atividade 1

Para realizar essa atividade, foi implementado o seguinte algoritmo:

- Definir a região de interesse (ROI) da imagem que queremos editar;
- Construir a imagem com a tatuagem destacada na cor branca (em escala de cinza);

- Varrer a ROI buscando pixeis branco;
- Construir um kernel centrado no pixel branco encontrado que seja constituído pelos valores dos pixeis vizinhos diferentes de preto;
- Definir o valor do pixel de saída da imagem tratada substituindo o pixel da imagem original pela média dos pixeis da vizinhança.

Seguindo a lógica deste algoritmo, inicialmente foi definida a região de interesse que gostaríamos de trabalhar. Esse foi um processo manual, onde usando o método pyplot dentro da biblioteca matplotlib buscou-se individualmente cada pixel.

Para definir a imagem auxiliar contendo a tatuagem em destaque, foi necessário primeiro encontrar as bordas da tatuagem. Em seguida, realizou-se uma limiarização sobre a imagem original e, por fim, aplicou-se uma operação lógica OR entre as duas, a fim de obter o máximo de detalhes possíveis da tatuagem. Nesta etapa, o algoritmo responsável por encontrar as bordas da tatuagem buscava identificar a magnitude da variação dos pixeis na imagem inteira e filtrado pelo processo de limiarização que retornava uma imagem binária. Para todos os processos de limiarização, foi definido, a partir de testes o melhor limiar que mantinha o nível de detalhamento necessário.

A Figura 2 mostra o processo realizado para conseguir destacar, com a maior exatidão possível os traços da tatuagem na figura original.

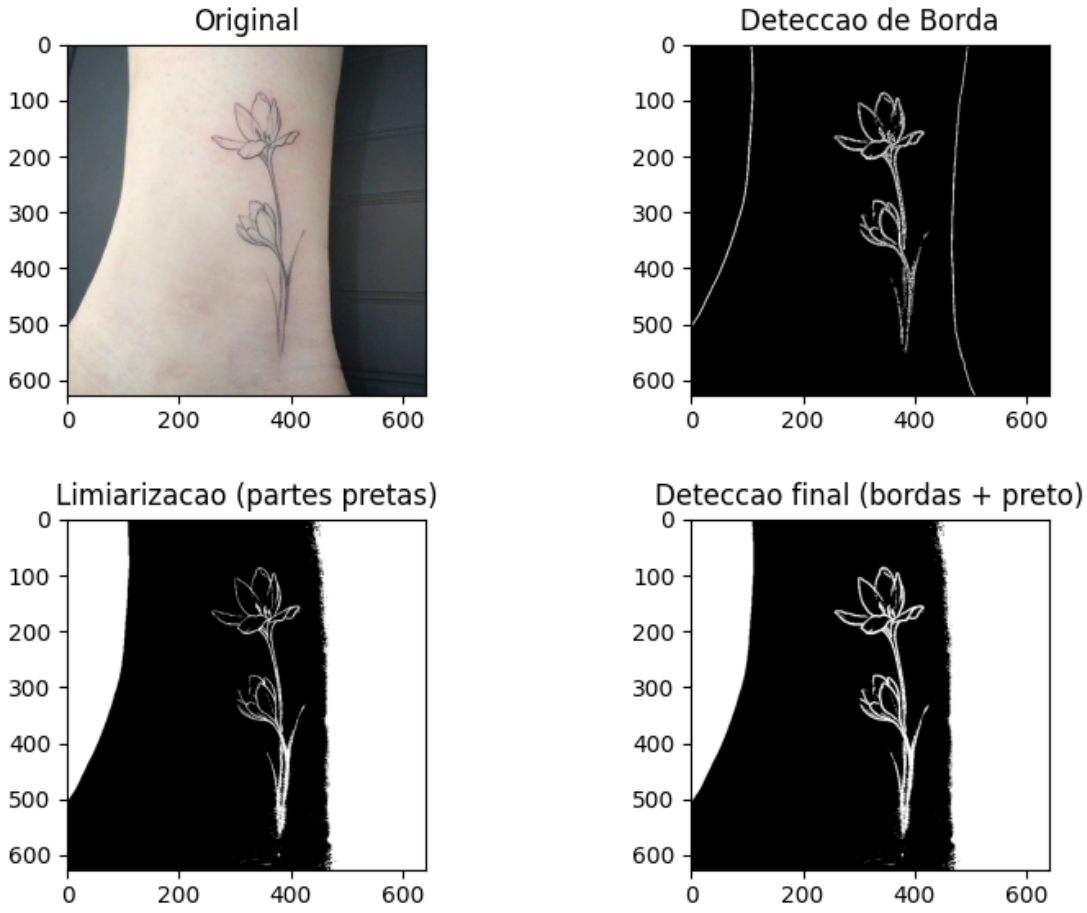


Figura 2: Processo de destaque dos traços da tatuagem.

Em seguida, tendo definido a ROI e destacado a tatuagem em uma imagem binária, onde o desenho estava em branco, varreu-se a ROI dentro da imagem binária buscando por pixeis na cor branca. Assim que se encontra esse pixel, é construído um kernel de 3 dimensões de tamanho 33x33 que armazena o valor dos vizinhos desse pixel, na imagem original, que tenham cor diferente de branco na imagem binária. Com o kernel preenchido foi realizado a média aritmética dos pixeis em cada camada para compor o pixel na imagem de saída (uma cópia da imagem original).

Com essa operação foi possível obter o resultado mostrado na Figura 3. O resultado obtido foi satisfatório mas não o ideal. Mesmo aplicando um kernel gaussiano para tentar suavizar os resíduos que sobraram, o algoritmo ainda não foi capaz de reduzir completamente os traços. Uma implementação que pode ser feita para tentar alcançar um resultado

mais satisfatório é substituir a média aritmética por uma média ponderada pela distância do pixel ao pixel central do kernel. Dessa maneira, os tons de pele mais próximo teriam peso maior na hora de compor a cor do pixel central.



Figura 3: Resultado do tratamento digital da imagem original.

Outro fator que se observou foi que o tamanho do kernel (33x33) gerou um aumento exponencial no processamento e, por consequência, na complexidade computacional. Devido aos 3 loops indentados percorrendo cada linha e coluna em cada camada da imagem original, quanto maior o kernel construído e maior o número de vizinhos que deveriam ser alcançados, mais se exigia computacionalmente. Para o caso do kernel 33x33, o tempo de processamento foi de aproximadamente 20 segundos.

Para justificar o uso de um kernel grande como o utilizado, foram testados tamanhos inferiores, como 3x3 e 9x9, mas com kernels assim, os pixeis ao redor do pixel central muitas vezes mantinham a cor branca, então o kernel era formado por poucos valores diferentes de 1, o que não era capaz de alterar o valor do pixel analisado, somente suavizar muito pouco o mesmo.

Seção 3: Descrição da Atividade 2:

O objetivo da Atividade 2 é implementar, em formato de função, o algoritmo de detecção de bordas de Canny, um dos métodos comumente utilizados em visão computacional na identificação de bordas de imagens. A atividade consiste em processar uma imagem em escala de cinza e obter uma imagem binária contendo as bordas detectadas. Para isso, o algoritmo deve seguir o fluxo apresentado na Figura 4.

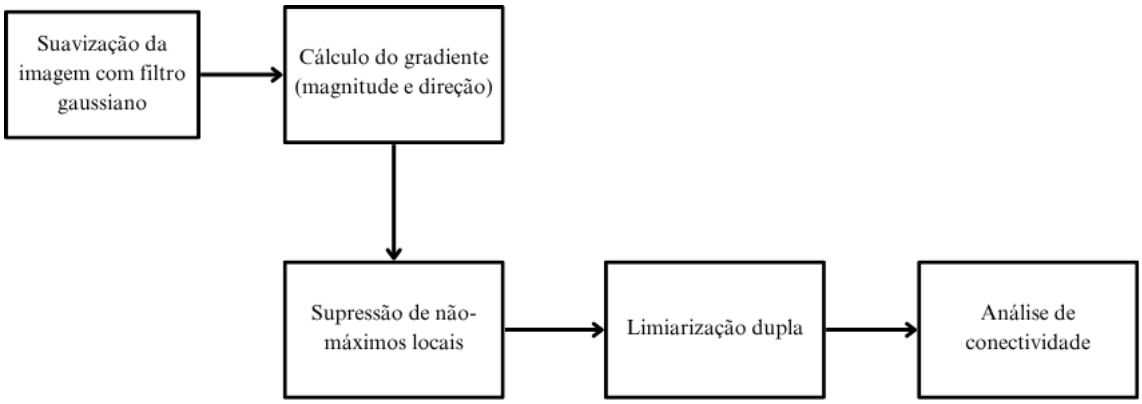


Figura 4: Fluxo do Algoritmo a ser implementado.

Ou seja:

1. Suavização da imagem com filtro gaussiano;
2. Cálculo do gradiente (magnitude e direção);
3. Supressão de não-máximos locais;
4. Limiarização dupla (classificação em bordas fortes e fracas);
5. Análise de conectividade, unindo as bordas fortes e as bordas fracas conectadas a elas.

Seção 3.1: Solução Proposta para a Atividade 2

O algoritmo foi desenvolvido passo a passo, conforme as instruções do roteiro. A seguir, cada etapa é descrita, acompanhada dos resultados visuais correspondentes.

Passo 1 - Suavização da imagem

A primeira etapa tem como objetivo a redução de ruído presente na imagem original. Para isso, a imagem foi filtrada por um kernel Gaussiano, com variância σ. Essa filtragem suaviza variações bruscas de intensidade e prepara a imagem para o cálculo do gradiente. A Figura 5 apresenta a imagem original e o resultado da suavização com $\omega = 5$ e $\sigma = \frac{(\omega-1)}{6}$. Observa-se que a diferença visual entre a imagem original e a suavizada é sutil.

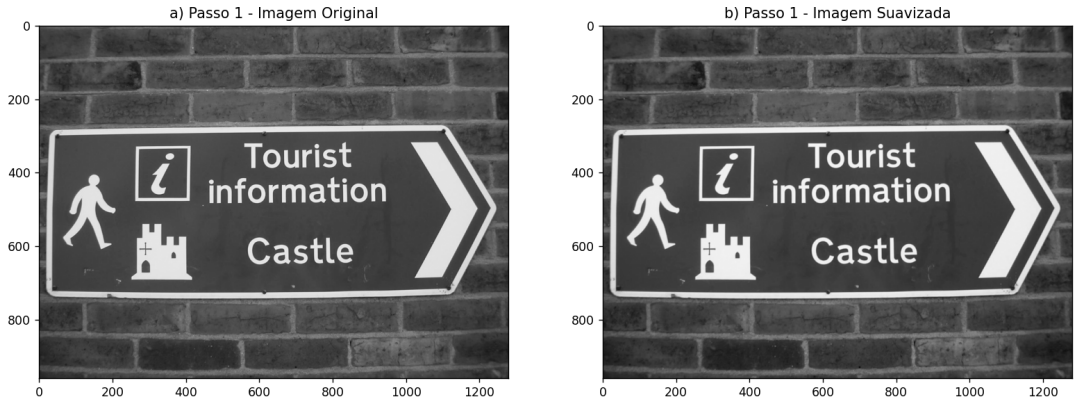


Figura 5: Resultado Passo 1 - (a) Imagem original. (b) Imagem suavizada.

Passo 2 - Cálculo do gradiente

Após a suavização, foram calculadas as derivadas parciais nas direções horizontal e vertical, gerando os gradientes G_x e G_y. Com base nesses valores, foram obtidas a magnitude e a direção do gradiente que indicam a intensidade e a orientação das possíveis bordas. A Figura 6 apresenta a magnitude e a fase do gradiente da imagem suavizada, enquanto a Figura 7 mostra separadamente os gradientes nas direções x e y.

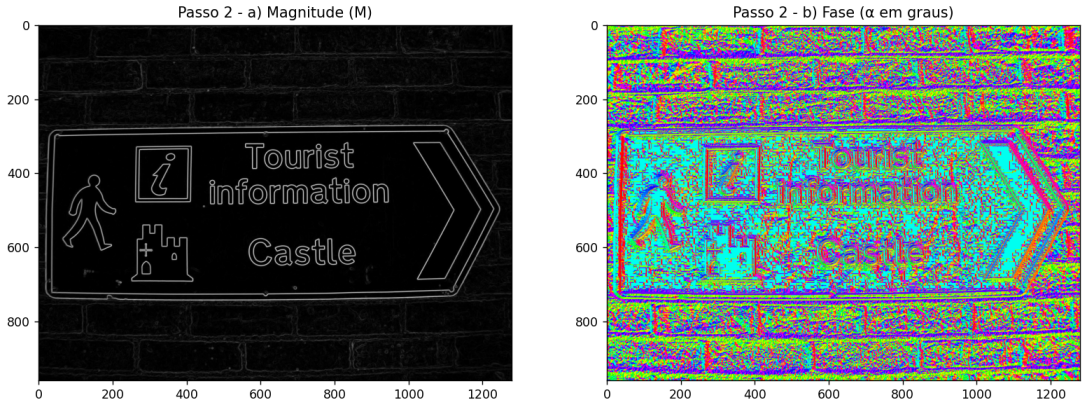


Figura 6: Resultado Passo 2 - (a) Magnitude do gradiente. (b) Direção do gradiente (α).

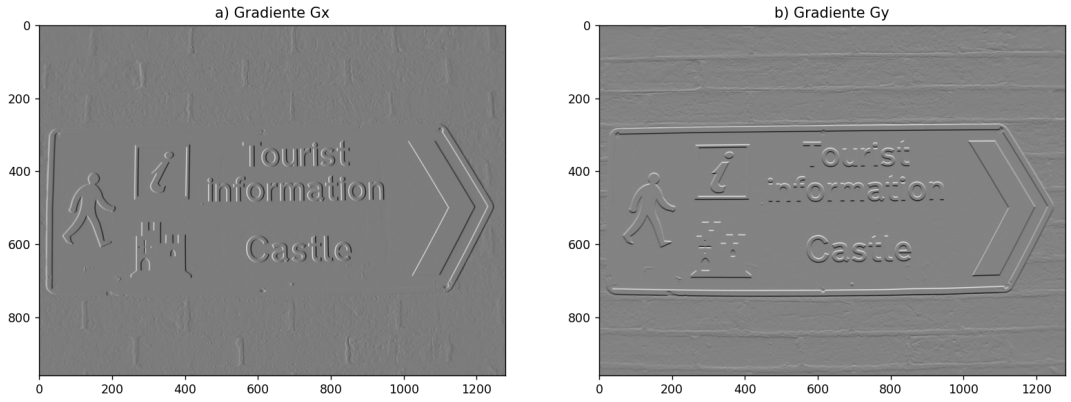


Figura 7: Resultado Passo 2 - (a) Gradiente G_x. (b) Gradiente G_y.

Passo 3 - Supressão de não máximos locais

A etapa de supressão de não-máximos locais tem como objetivo refinar as bordas, tornando-as mais finas e precisas, de tal forma que apenas os pixeis que representam o ponto máximo da intensidade na direção da borda sejam mantidos. Primeiramente, os ângulos negativos foram convertidos em positivos pela soma de 180°, e a direção do gradiente em cada pixel foi quantizada em quatro direções principais:

- 0° (horizontal);
- 45° (diagonal ascendente);

- 90° (vertical);
- 135° (diagonal descendente).

Para cada pixel da imagem, foram identificados dois vizinhos localizados na direção do gradiente correspondente. Após isso, foi comparado a magnitude do pixel central com a dos dois vizinhos. Caso o pixel central possuísse magnitude maior ou igual às magnitudes dos vizinhos, ele era mantido como pixel de borda. Caso contrário, ele recebe valor zero, por não representar um pixel de borda. A Figura 8 apresenta o resultado dessa etapa, onde as bordas se tornam mais definidas.

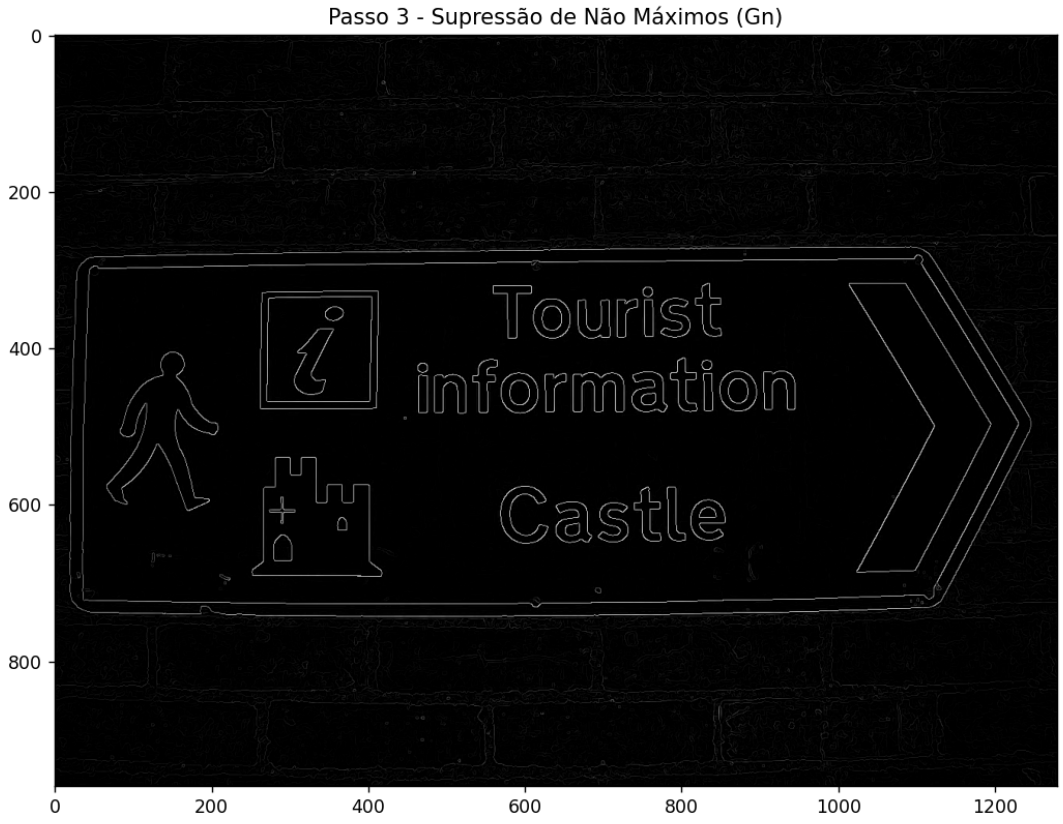


Figura 8: Resultado Passo 3 - Supressão de não-máximos locais G_n.

Passo 4 - Limiarização dupla

Após a supressão de não-máximos locais, a imagem resultante contém pixeis com intensidades diferentes, representando possíveis bordas, mas ainda não há distinção clara entre

bordas reais e bordas falsas. Para resolver isso, foi aplicado o processo de limiarização dupla, cujo objetivo é classificar os pixeis de borda de acordo com sua força. Para obtenção dos resultados apresentados dois limiares foram definidos, $\tau_H = 90$, limiar superior, que determina as bordas mais intensas e o $\tau_L = 30$, limiar inferior, que identifica as bordas de menor intensidade. Os pixeis com valor maior ou igual ao limiar superior são classificados como bordas fortes e recebem intensidade máxima (255) na imagem GNH (Figura 9a). Já os pixeis com valor maior ou igual ao limiar inferior compõem a imagem GNL (Figura 9b), que inclui tanto bordas fortes quanto bordas fracas. Em seguida, calcula-se $GNL = GNL - GNH$, que representa somente as bordas fracas, já que as bordas fortes são subtraídas da imagem de limiar inferior, o resultado desse cálculo é apresentado na Figura 10. Essa operação é fundamental para separar os dois tipos de borda e preparar o processo de análise de conectividade que virá em seguida.

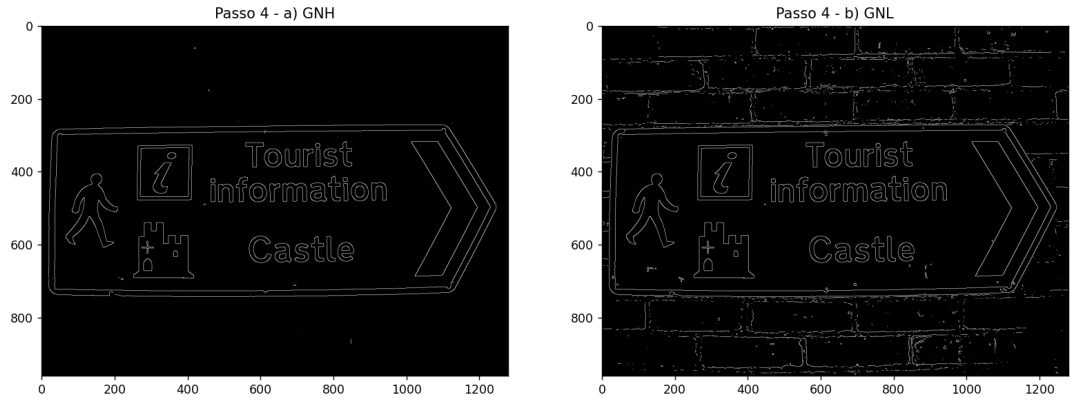


Figura 9: Resultado Passo 4 - (a) Imagem G_{NH}. (b) Imagem G_{NL}.

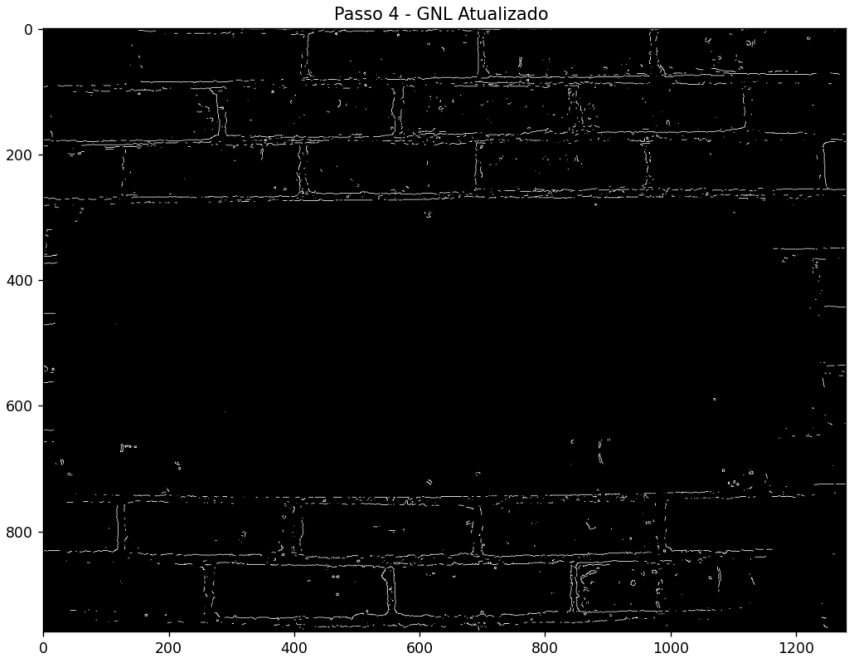


Figura 10: Resultado Passo 4 - Imagem G_{NL} atualizada.

Passo 5 – Análise de Conectividade

A última etapa do algoritmo, verifica a conectividade entre bordas fortes e fracas para decidir quais bordas fracas devem ser mantidas na imagem final. Essa etapa busca garantir que as bordas contínuas e coerentes sejam preservadas, buscando eliminar ruídos e pequenas descontinuidades que poderiam comprometer o contorno da imagem. O processo foi implementado de forma iterativa, primeiramente uma cópia da imagem GNH foi criada, denominada Ib, contendo inicialmente apenas as bordas fortes. Para cada pixel de borda forte (valor 255) em GNH, o algoritmo analisa sua vizinhança 3×3 na imagem GNL. Caso algum dos pixeis vizinhos na vizinhança pertença a uma borda fraca (valor 255 em GNL), ele é colocado como borda válida e também recebe valor 255 na imagem final Ib. Após percorrer todos os pixeis de borda forte, as bordas validadas de GNL são somadas à imagem GNH, resultando na imagem final Ib, conforme apresentado na Figura 11.

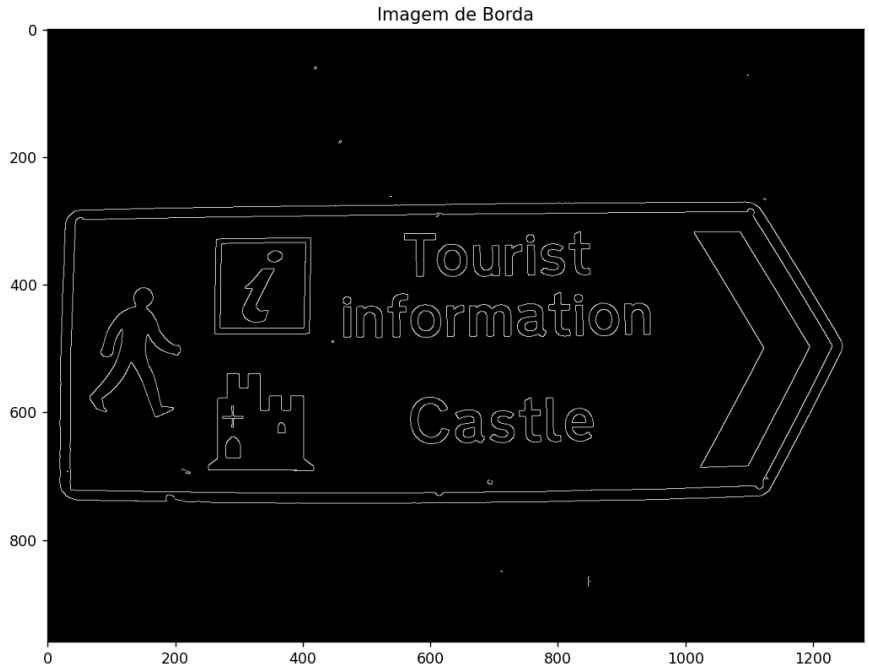


Figura 11: Resultado Passo 5 - Imagem de Borda.

Utilizando os mesmos parâmetros de limiarização (τ_L e τ_H), foi executada a função *cv2.Canny()* do OpenCV para fins de comparação. O resultado, apresentado na Figura 12, mostra que a escolha de limiares influencia fortemente o resultado da detecção. Observa-se que, com os mesmos valores utilizados na implementação manual, a função do OpenCV produz uma imagem mais ruidosa, com detecção de bordas adicionais nas regiões de fundo. Ao ajustar os valores para $\tau_H = 200$ e $\tau_L = 50$, o resultado é significativamente aprimorado, como é apresentado na Figura 13, apresentando bordas mais nítidas e limpas, com melhor definição dos contornos principais e redução de ruído. Essa comparação reforça a importância da adequada seleção dos limiares, que deve ser feita de forma adaptada às características da imagem e à implementação utilizada.

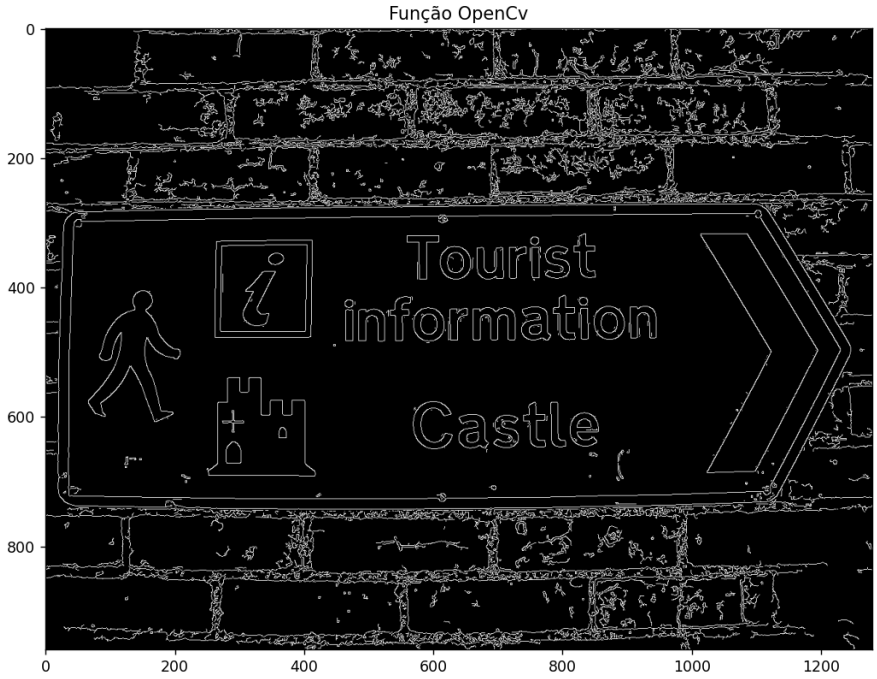


Figura 12: Resultado Função do OpenCV.

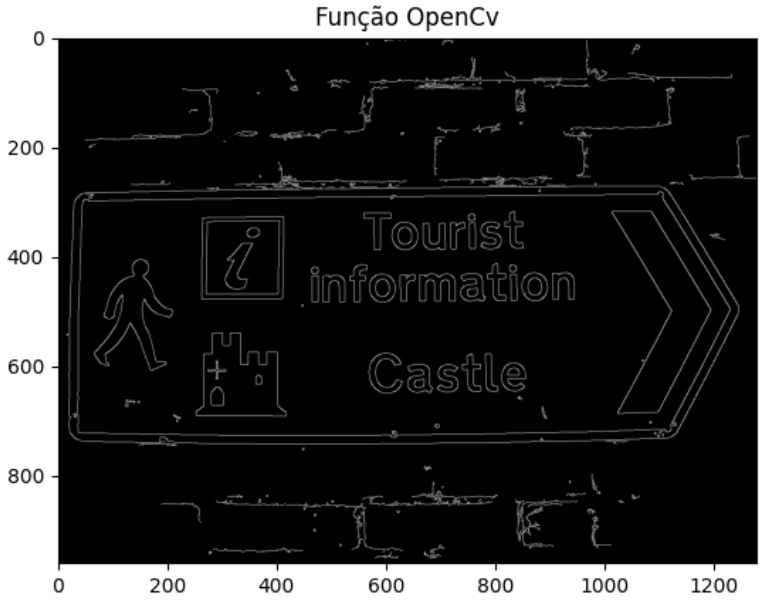


Figura 13: Resultado Função do OpenCV com $\tau_H = 200$ e $\tau_L = 50$.

A implementação manual do algoritmo de Canny apresentou resultados satisfatórios na detecção de bordas. A filtragem Gaussiana reduziu o ruído, preservando as estruturas

principais. O cálculo do gradiente destacou as regiões de variação de intensidade, e a supressão de não-máximos refinou as bordas. A limiarização dupla permitiu separar bordas fortes e fracas, e a análise de conectividade garantiu a preservação de contornos contínuos, resultando em uma imagem final limpa e bem definida. Na comparação com a função *cv2.Canny()*, observou-se comportamento semelhante, embora o método do OpenCV tenha exibido maior sensibilidade a escolha do τ_L e τ_H. Ajustes nos limiares melhoraram significativamente o resultado, evidenciando a importância da escolha adequada desses parâmetros.